SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA0620	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	Kamalesh Thirunavukkarasu	**Reg. No.:**	192424228
Section / Batch:	2024	**Date:**	16/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – APPLICATION BASED QUESTIONS

Q1:Product Sorting System

A system compares sorting algorithms with complexities O(n log n) and O(n²). Analyze both using
asymptotic notation and compare their growth rates. Explain which algorithm is more efficient for
large datasets. Justify your answer based on scalability. Interpret efficiency clearly.

Q2:Cloud Storage Processing System

A cloud storage system processes data using the recurrence T(n) = T(n/2) + n. Apply the substitution method to expand the recurrence. Derive the final time complexity and simplify it.
Analyze how performance scales with increasing data size. Interpret efficiency using asymptotic
notation.

Code of Conduct

I Kamalesh Thirunavukkarasu(192424228) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Kamalesh Thirunavukkarasu*_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.Product Sorting System – Comparative Analysis of O(n log n) and O(n²) Using Asymptotic Notation

---

Problem Statement

A product sorting system compares two sorting algorithms having time complexities **O(n log n)** and **O(n²)**. Analyze both algorithms using asymptotic notation, compare their growth rates, determine which algorithm is more efficient for large datasets, justify your answer based on scalability, and interpret the efficiency clearly.

Introduction

Sorting is one of the most fundamental operations in computer science and software engineering. It is the process of arranging data in a specific order, such as ascending or descending. Product sorting is widely used in e-commerce platforms, inventory management systems, cloud databases, banking applications, search engines, and recommendation systems. Whenever a customer selects options such as **Sort by Price**, **Sort by Rating**, **Sort by Popularity**, or **Sort by Newest**, the underlying system executes a sorting algorithm.

The efficiency of a sorting algorithm directly affects the response time of the application. As the number of products increases from hundreds to millions, choosing the right sorting algorithm becomes extremely important. Algorithms with poor time complexity become slow and consume more computational resources, while efficient algorithms maintain fast performance even for very large datasets.

To evaluate the performance of sorting algorithms independent of hardware and programming language, **asymptotic notation** is used. Asymptotic analysis studies how the running time of an algorithm grows with increasing input size.

In this problem, two algorithms with time complexities **O(n²)** and **O(n log n)** are compared to determine which is more suitable for processing large product datasets.

Step 1: Understanding Asymptotic Notation

Asymptotic notation is a mathematical tool used to describe the performance of algorithms for very large input sizes.

It ignores:

- Machine specifications
- Programming language
- Compiler optimizations
- Constant factors

and focuses only on the growth rate of the algorithm.

There are three major asymptotic notations.

---

1. Big-O Notation (O)

Big-O notation represents the **upper bound** or **worst-case running time** of an algorithm.

It answers the question:

"What is the maximum amount of time the algorithm may take?"

Examples:

- Bubble Sort → $O(n^2)$
- Selection Sort → $O(n^2)$
- Merge Sort → $O(n \log n)$

---

2. Big-Omega Notation (Ω)

Big-Omega notation represents the **best-case running time**.

It indicates the minimum amount of work the algorithm must perform.

Examples:

- Bubble Sort (Already Sorted) → $\Omega(n)$
- Merge Sort → $\Omega(n \log n)$

---

3. Theta Notation (Θ)

Theta notation represents the **tight bound** or **average-case complexity** when the upper and lower bounds are the same.

Examples:

- Merge Sort → $\Theta(n \log n)$
- Heap Sort → $\Theta(n \log n)$
- Selection Sort → $\Theta(n^2)$

Step 2: Understanding $O(n^2)$ Sorting Algorithm

Algorithms with quadratic complexity perform comparisons between almost every pair of elements.

Examples include

- Bubble Sort
- Selection Sort

- Insertion Sort (Worst Case)

Working Principle

Suppose there are **n products**.

For every product,

the algorithm compares it with almost every remaining product.

Total comparisons become

n(n−1)2\frac{n(n-1)}{2}2n(n−1)

Ignoring constants,

T(n)=O(n2)T(n)=O(n^2)T(n)=O(n2)

Example

Suppose there are **5 products**

Laptop
Mobile
Camera
Watch
Headphone

Bubble Sort performs comparisons like

Laptop ↔ Mobile
Laptop ↔ Camera
Laptop ↔ Watch
Laptop ↔ Headphone

Mobile ↔ Camera
Mobile ↔ Watch
Mobile ↔ Headphone

Camera ↔ Watch
Camera ↔ Headphone

Watch ↔ Headphone

Nearly every product is compared with every other product.

Hence,

Number of operations grows approximately as

n2n^2n2

Characteristics

- Easy to implement
- Requires little memory
- Suitable for educational purposes
- Efficient only for small datasets
- Poor scalability

Step 3: Understanding O(n log n) Sorting Algorithm

Algorithms with **O(n log n)** complexity use the **Divide and Conquer** strategy.

Examples include

- Merge Sort
- Heap Sort
- Quick Sort (Average Case)

Working Principle

The algorithm repeatedly divides the products into two equal halves.

Example:

Suppose there are **16 products**

16

↓

8 + 8

↓

4 + 4 + 4 + 4

↓

2 + 2 + 2 + 2 + 2 + 2 + 2 + 2

↓

1 + 1 + 1 + 1 + ...

The number of divisions is

log⁡2n\log_2 nlog2 n

Each level processes all **n** products.

Therefore,

T(n)=nlog⁡nT(n)=n\log nT(n)=nlogn

Characteristics

- High efficiency
- Better scalability
- Suitable for millions of records
- Used in industrial software
- Faster than quadratic algorithms

Step 4: Mathematical Comparison

Suppose there are

n=1000n=1000n=1000

Quadratic Algorithm

Operations

10002=1,000,0001000^2 = 1,000,00010002=1,000,000

Approximately one million comparisons.

Log Linear Algorithm

Operations

1000×log⁡2(1000)1000\times\log_2(1000)1000×log2 (1000)

Since

log⁡2(1000)≈10\log_2(1000)\approx10log2 (1000)≈10

Operations become

1000×10=10,0001000\times10 = 10,0001000×10=10,000

Only ten thousand comparisons.

Difference

Quadratic Algorithm

1,000,000 Comparisons

Log Linear Algorithm

10,000 Comparisons

The improvement is enormous.

Step 5: Growth Rate Comparison

Number of Products (n)	$n \log_2 n$	n^2
10	33	100
100	664	10,000
500	4,483	250,000
1,000	9,966	1,000,000
5,000	61,438	25,000,000
10,000	132,877	100,000,000
50,000	780,482	2,500,000,000
100,000	1,660,964	10,000,000,000
1,000,000	19,931,568	1,000,000,000,000

Observation

Initially both algorithms seem acceptable.

As the number of products increases,

the value of

n2n^2n2

grows dramatically.

Whereas

nlog⁡nn\log nnlogn

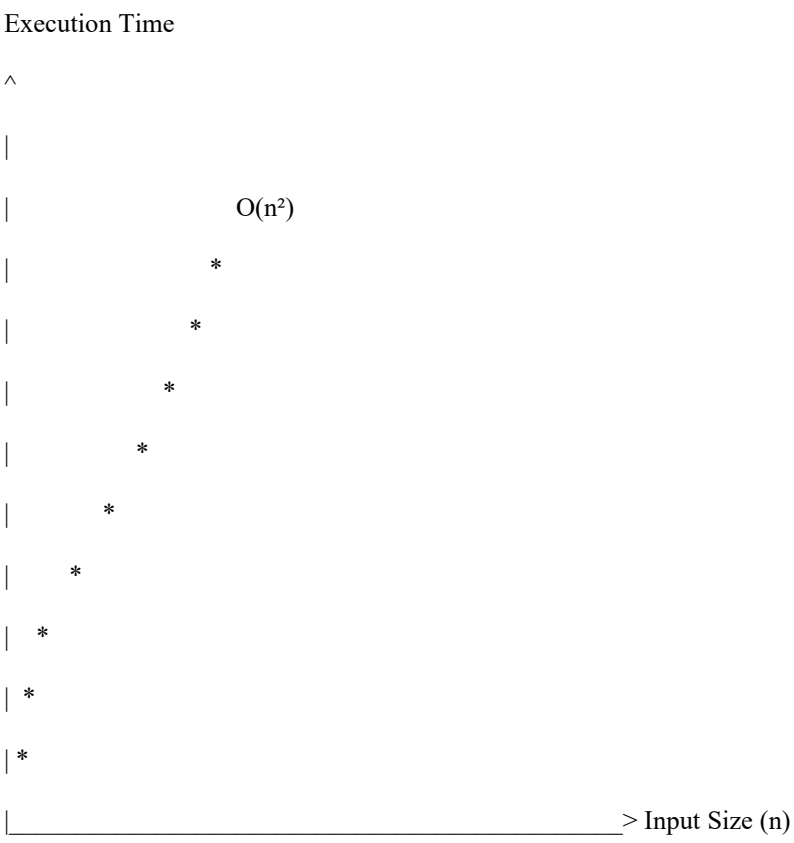
grows much more slowly.

Therefore,

O(nlog⁡n)<O(n2)O(n\log n) < O(n^2)O(nlogn)<O(n2)

for very large values of **n**.

Step 6: Graphical Interpretation



The graph shows that

- $O(n^2)$ increases rapidly.
- O(n log n) grows much more slowly.

Hence,

O(n log n) is more scalable.

Step 7: Scalability Analysis

Scalability measures how efficiently an algorithm performs when the input size increases.

Case 1 – $O(n^2)$

If the number of products doubles,

from

nnn

to

2n2n2n

then

(2n)2=4n2(2n)^2 = 4n^2(2n)2=4n2

Execution time becomes approximately **four times larger**.

Example

1000 Products

↓

1 Million Operations

2000 Products

↓

4 Million Operations

The increase is very high.

Case 2 – O(n log n)

If products double,

Execution becomes

2nlog⁡(2n)2n\log(2n)2nlog(2n)

Only slightly more than double.

Example

1000 Products

↓

≈10,000 Operations

2000 Products

↓

≈22,000 Operations

Growth remains manageable.

Therefore,

O(n log n)

is highly scalable.

Step 8: Performance Analysis for Large Product Databases

Suppose an online shopping website contains

- Electronics
- Mobiles
- Laptops
- Watches
- Shoes
- Clothes

Every customer requests sorting based on

- Price
- Rating
- Discount
- Popularity
- New Arrival

The sorting algorithm executes every time a user changes the sorting option.

Using $O(n^2)$

The server performs millions or billions of comparisons.

Result

- Slow loading
- Increased CPU usage
- High memory utilization
- Poor customer experience

Using O(n log n)

The server performs significantly fewer comparisons.

Result

- Fast response
- Better scalability
- Lower processing cost
- Improved user satisfaction

- Suitable for cloud deployment

Step 9: Real-Time Applications

Real-Time Application	Role of Sorting
E-commerce Websites	Sort products by price, rating, popularity
Banking Systems	Sort customer transactions
Hospital Management	Sort patient records
Library Systems	Sort books alphabetically
Search Engines	Rank search results
Cloud Storage	Sort file metadata
Inventory Management	Arrange stock by quantity
Online Ticket Booking	Sort available seats
AI & Machine Learning	Sort datasets before preprocessing
Database Systems	Sort records before indexing

Step 10: Advantages and Disadvantages

Advantages of O(n log n)

- Faster execution
- Better scalability
- Suitable for big data
- Used in enterprise software
- Lower computational cost
- Efficient memory usage (algorithm dependent)
- Supports parallel processing

Disadvantages of $O(n^2)$

- Large number of comparisons
- Slow execution
- High CPU utilization
- Poor scalability
- Not suitable for millions of records

Step 11: Asymptotic Interpretation

Notation	Meaning	$O(n^2)$	$O(n \log n)$
O	Worst Case	$O(n^2)$	$O(n \log n)$
Ω	Best Case	$\Omega(n)$ (Bubble Sort)	$\Omega(n \log n)$
Θ	Tight Bound	$\Theta(n^2)$	$\Theta(n \log n)$

Step 12: Final Comparison

Feature	$O(n^2)$	$O(n \log n)$
Growth Rate	Very Fast	Moderate
Comparisons	Very High	Much Lower
Scalability	Poor	Excellent
Performance	Slow	Fast
Large Datasets	Not Suitable	Highly Suitable
Industrial Usage	Rare	Very Common
Examples	Bubble Sort, Selection Sort	Merge Sort, Heap Sort

Conclusion

The comparative analysis demonstrates that algorithms with **$O(n \log n)$** complexity are significantly more efficient than those with **$O(n^2)$** complexity for sorting large product datasets. Although quadratic algorithms are simple to understand and implement, their execution time increases rapidly as the number of products grows, making them unsuitable for modern applications that process thousands or millions of records. In contrast, **$O(n \log n)$** algorithms such as Merge Sort and Heap Sort divide the problem into smaller subproblems and combine the results efficiently, resulting in much lower execution time and better scalability. Consequently, modern e-commerce platforms, cloud storage systems, banking applications, and database management systems widely adopt **$O(n \log n)$** algorithms because they provide faster processing, efficient resource utilization, and excellent performance for large-scale datasets.

2.Cloud Storage Processing System – Substitution Method

Problem Statement

A cloud storage system processes data using the recurrence:

T(n)=T(n2)+nT(n)=T\left(\frac{n}{2}\right)+nT(n)=T(2n)+n

Apply the **substitution (expansion) method** to expand the recurrence, derive the final time complexity, simplify the recurrence, analyze how performance scales with increasing data size, and interpret the efficiency using **asymptotic notation**.

Introduction

Cloud storage systems such as **Google Drive, Dropbox, Microsoft OneDrive, Amazon S3, and iCloud** process enormous amounts of data every second. Large files are usually divided into smaller blocks or chunks before being uploaded, synchronized, compressed, encrypted, or replicated across multiple servers. Processing these blocks independently improves efficiency and allows cloud systems to scale effectively.

Many cloud storage algorithms follow the **Divide-and-Conquer** strategy, where a large problem is divided into smaller subproblems, each processed separately, and the results are combined. The recurrence

T(n)=T(n2)+nT(n)=T\left(\frac{n}{2}\right)+nT(n)=T(2n)+n

represents such a process. Here, the data size is reduced by half at every recursive call, while linear work is performed at each level for processing, synchronization, or merging. The substitution method is used to solve this recurrence and determine its asymptotic time complexity.

Step 1: Understand the Recurrence

The given recurrence is

T(n)=T(n2)+nT(n)=T\left(\frac{n}{2}\right)+nT(n)=T(2n)+n

It consists of two parts:

- T(n/2)T(n/2)T(n/2) : The file is divided into two equal parts, but only one recursive branch is processed further.
- nnn : Linear work performed at the current level, such as reading, encrypting, compressing, or synchronizing the data.

Example

Suppose a cloud storage system processes **16 MB** of data.

The processing follows:

16 MB
↓

8 MB
↓

4 MB
↓

2 MB
↓

1 MB

At every level,

- the file size becomes half,
- but processing proportional to the current data size is still required.

Thus, every recursive call performs additional work equal to the current input size.

Step 2: Expand the Recurrence (Substitution Method)

Start with the given recurrence

$$T(n)=T(n2)+nT(n)=T\left(\frac{n}{2}\right)+nT(n)=T(2n\ )+n$$

First Expansion

Expand

$$T(n2)T\left(\frac{n}{2}\right)T(2n\ )$$

Substitute

$$T(n2)=T(n4)+n2T\left(\frac{n}{2}\right)=T\left(\frac{n}{4}\right)+\frac{n}{2}T(2n\ )=T(4n\ )+2n$$

Therefore,

$$T(n)=T(n4)+n2+nT(n)=T\left(\frac{n}{4}\right)+\frac{n}{2}+nT(n)=T(4n\ )+2n\ +n$$

Second Expansion

Expand

$$T(n4)T\left(\frac{n}{4}\right)T(4n\ )$$

Substitute

$$T(n4)=T(n8)+n4T\left(\frac{n}{4}\right)=T\left(\frac{n}{8}\right)+\frac{n}{4}T(4n\ )=T(8n\ )+4n$$

Hence,

T(n)=T(n8)+n4+n2+nT(n)=T\left(\frac{n}{8}\right)+\frac{n}{4}+\frac{n}{2}+nT(n)=T(8n)+4n +2 n +n

---

Third Expansion

Expand

T(n8)T\left(\frac{n}{8}\right)T(8n)

Substitute

T(n8)=T(n16)+n8T\left(\frac{n}{8}\right)=T\left(\frac{n}{16}\right)+\frac{n}{8}T(8n)=T(16n)+8 n

Therefore,

T(n)=T(n16)+n8+n4+n2+nT(n)=T\left(\frac{n}{16}\right)+\frac{n}{8}+\frac{n}{4}+\frac{n}{2}+nT(n) =T(16n)+8n +4n +2n +n

---

Step 3: Observe the Pattern

After every expansion,

- The problem size becomes half.
- A new term is added.

The recurrence becomes

T(n)=T(n2k)+n(1+12+14+⋯ +12k−1)T(n)=T\left(\frac{n}{2^k}\right) +n \left(1+\frac12+\frac14+\cdots+\frac1{2^{k-1}} \right)T(n)=T(2kn)+n(1+21 +41 +⋯ +2k−11)

This is the general expansion.

Step 4: Find When the Recursion Stops

The recursion stops when

n2k=1\frac{n}{2^k}=12kn =1

Multiply both sides by 2k2^k2k

n=2kn=2^kn=2k

Taking logarithm (base 2),

k=log⁡2nk=\log_2 nk=log2 n

Hence,

the recursion contains

log⁡n\boxed{\log n}logn

levels.

Step 5: Substitute the Value of kkk

Substitute

k=log⁡nk=\log nk=logn

The recurrence becomes

T(n)=T(1)+n(1+12+14+⋯+1n)T(n) = T(1) + n \left(1+\frac12+\frac14+\cdots+\frac1n \right)T(n)=T(1)+n(1+21 +41 +⋯+n1)

Since

T(1)=1T(1)=1T(1)=1

is constant,

only the geometric series remains.

Step 6: Simplify the Geometric Series

The summation is

1+12+14+18+⋯1+\frac12+\frac14+\frac18+\cdots1+21 +41 +81 +⋯

This is a geometric progression.

Its sum is

2−1n2-\frac1n2−n1

Therefore,

T(n)=1+n(2−1n)T(n) = 1+n\left(2-\frac1n\right)T(n)=1+n(2−n1)

Simplifying,

T(n)=1+2n−1T(n) = 1+2n-1T(n)=1+2n−1

Hence,

T(n)=2nT(n)=2nT(n)=2n

Ignoring constants,

T(n)=Θ(n)\boxed{T(n)=\Theta(n)}T(n)=Θ(n)

Step 7: Final Time Complexity

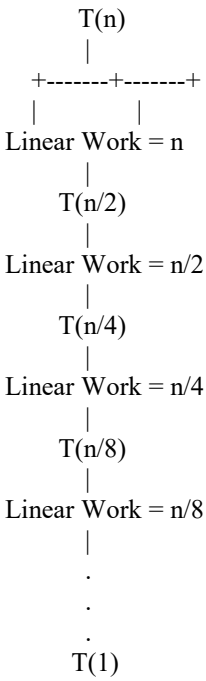
Therefore,

T(n)=Θ(n)\boxed{T(n)=\Theta(n)}T(n)=Θ(n)

Hence,

- **Worst Case = O(n)**
- **Best Case = Ω(n)**
- **Tight Bound = Θ(n)**

Recursion Tree



Observation

At every level,

- Problem size becomes half.

- Processing work also becomes half.
- Total work forms a geometric series.

The accumulated work is

n+n2+n4+⋯ n+\frac n2+\frac n4+\cdotsn+2n +4n +⋯

which equals

2n2n2n

Therefore,

overall complexity becomes

Θ(n)\Theta(n)Θ(n)

Performance Analysis for Large Cloud Storage Systems

Suppose a cloud storage server processes

- Images
- Videos
- Documents
- Database backups
- Virtual machine snapshots

Each file is divided into smaller blocks.

Each block is

- encrypted,
- compressed,
- synchronized,
- replicated,
- uploaded.

Since the file size is reduced by half after every recursive call, the number of recursive levels increases slowly.

However,

each level still processes data proportional to the current block size.

Consequently,

the total work becomes proportional to the original file size.

Example

Suppose

1,000 MB

of data is processed.

Work performed

≈1000 units

Suppose

10,000 MB

of data is processed.

Work becomes

≈10,000 units

Suppose

100,000 MB

of data is processed.

Work becomes

≈100,000 units

Thus,

processing time grows approximately in direct proportion to the input size.

This is called **linear scalability**.

Scalability Analysis

Linear algorithms scale efficiently.

If the input size doubles,

processing time approximately doubles.

Example

Data Size	Processing Time
1 GB	T
2 GB	2T
5 GB	5T
10 GB	10T
50 GB	50T
100 GB	100T

Unlike quadratic algorithms,

there is no exponential increase in processing time.

Hence,

the recurrence is highly scalable.

Real-Time Applications

Real-Time Application	How Divide-and-Conquer is Used
Cloud Storage	Split files into blocks and process each block
Google Drive Synchronization	Divide files and synchronize chunks
Dropbox	Upload files block by block
Amazon S3	Multipart upload processing
Distributed File Systems	Store data across multiple servers
Hadoop	Divide files before processing
Data Compression	Compress smaller blocks independently
Backup Systems	Split large backups into chunks
Video Streaming	Divide videos into segments
Content Delivery Networks (CDN)	Process data in distributed chunks

---

Advantages

- Efficient for very large files
- Linear time complexity
- Excellent scalability
- Lower memory overhead
- Suitable for distributed processing
- Supports parallel execution
- Ideal for cloud computing
- Faster synchronization
- Reduced processing bottlenecks

Disadvantages

- Recursive implementation may introduce function-call overhead.
- Processing cannot be completed until recursive calls finish.
- Large recursion depth may require additional stack memory (though only logarithmic depth).

Asymptotic Interpretation

Notation	Meaning	Result
O(n)	Upper Bound	Execution time does not exceed linear growth.
Ω(n)	Lower Bound	At least linear work is required.
Θ(n)	Tight Bound	Exact asymptotic growth is linear.

Conclusion

The recurrence T(n)=T(n/2)+nT(n)=T(n/2)+nT(n)=T(n/2)+n represents a divide-and-conquer process commonly used in cloud storage systems, where the input is repeatedly divided into smaller parts while performing linear work at each recursive level. Using the substitution method, the recurrence expands into a geometric series whose total sum is proportional to **2n**, which simplifies to **Θ(n)** after ignoring constant factors. This indicates that the execution time grows linearly with the size of the input data. As the amount of stored data increases, the processing time also increases proportionally, making the algorithm highly efficient, predictable, and scalable. Such linear-time algorithms are widely used in modern cloud computing environments because they can efficiently handle large datasets while maintaining good performance.

Final Answer

Given the recurrence

T(n)=T(n2)+nT(n)=T\left(\frac{n}{2}\right)+nT(n)=T(2n)+n

Using the substitution method,

T(n)=T(n2k)+n(1+12+14+⋯ +12k−1)T(n)=T\left(\frac{n}{2^k}\right)+n\left(1+\frac12+\frac14+\cdots +\frac1{2^{k-1}}\right)T(n)=T(2kn)+n(1+21 +41 +⋯ +2k−11)

The recursion stops when

n2k=1\frac{n}{2^k}=12kn =1

which gives

k=log⁡nk=\log nk=logn

The geometric series simplifies to

2−1n2-\frac1n2−n1

Hence,

T(n)=2n=Θ(n)T(n)=2n=\Theta(n)T(n)=2n=Θ(n)

Therefore,

- **Worst Case = O(n)**
- **Best Case = Ω(n)**
- **Tight Bound = Θ(n)**

Thus, the cloud storage processing system runs in **linear time**, making it highly efficient and scalable for processing large files, distributed storage, cloud synchronization, backup systems, and modern data-intensive applications